NumPy in Python

what is Numpy in python explain with code

Sure! Let's go **in-depth** about **NumPy** in Python. I'll explain theory first, then give examples.

What is NumPy?

- **NumPy** stands for **Numerical Python**.
- It is a **powerful library for numerical computing** in Python.
- Main feature: **N-dimensional arrays** (`ndarray`) that are faster and more efficient than Python lists.
- Provides **mathematical functions**, **linear algebra**, **statistics**, **random number generation**, and more.
- Widely used in **data science, machine learning, and scientific computing**.

Why use NumPy instead of Python lists?

Feature	Python List	NumPy Array
Speed	Slower	Faster (implemented in C)
Memory	More memory	Less memory
Operations	Element-wise operations require loops	Supports vectorized operations (fast)
Functions	Limited math operations	Hundreds of math and linear algebra functions

---

Basic NumPy Usage

First, you need to **import NumPy**:

```
Python
```

```python
import numpy as np  # np is the common alias
```

---

1. Creating NumPy Arrays

```python
import numpy as np

# From a Python list
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)          # [1 2 3 4 5]
print(type(arr1))     # <class 'numpy.ndarray'>

# 2D Array (matrix)
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

---

2. Array Attributes

```python
print(arr2.shape)   # (2, 3) → 2 rows, 3 columns
print(arr2.size)    # 6 → total elements
print(arr2.dtype)   # data type, e.g., int32
```

---

3. Creating Arrays with Functions

```python
# Array of zeros
zeros = np.zeros((2, 3))
print(zeros)

# Array of ones
ones = np.ones((3, 2))
print(ones)
```

```python
# Array of a range of numbers
arr_range = np.arange(0, 10, 2)  # start, stop, step
print(arr_range)

# Array with evenly spaced values
linspace_arr = np.linspace(0, 1, 5)  # 5 numbers from 0 to 1
print(linspace_arr)
```

---

4. Basic Array Operations

```python
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Arithmetic operations
print(a + b)  # [5 7 9]
print(a - b)  # [-3 -3 -3]
print(a * b)  # [4 10 18] → element-wise multiplication
print(a / b)  # [0.25 0.4 0.5]
print(a ** 2) # [1 4 9] → element-wise power

# Vectorized operations are faster than loops
```

---

5. Useful NumPy Functions

```python
arr = np.array([1, 2, 3, 4, 5])

print(np.sum(arr))       # Sum of elements → 15
print(np.mean(arr))      # Mean → 3.0
print(np.std(arr))       # Standard deviation
print(np.max(arr))       # Maximum → 5
print(np.min(arr))       # Minimum → 1
```

---

6. Indexing and Slicing

```python
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])    # 10 → first element
print(arr[-1])   # 50 → last element
```

```
print(arr[1:4])  # [20 30 40] → slicing
print(arr[:3])   # [10 20 30] → first three elements
```

2D array indexing:

```python
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])

print(arr2[0, 1])  # 2 → first row, second column
print(arr2[:, 1])  # [2 5 8] → entire second column
print(arr2[1, :])  # [4 5 6] → second row
```

---

7. Reshaping Arrays

```python
arr = np.arange(1, 10)  # [1 2 3 4 5 6 7 8 9]
reshaped = arr.reshape(3, 3)
print(reshaped)
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

✅ Summary

- NumPy is **essential for numerical computing** in Python.
- It provides **fast arrays**, **vectorized operations**, and **mathematical functions**.
- Key features: **array creation, indexing, reshaping, math operations, linear algebra**.

If you want, I can also make a **small demo comparing NumPy vs Python lists in speed** so you can see why NumPy is faster.

Do you want me to do that?

various ffrequently used numpy method give me list

Absolutely! Here's a **comprehensive list of frequently used NumPy methods** grouped by functionality for easy reference. I'll also give a **short explanation** for each.

---

1. Array Creation

Method	Description
`np.array()`	Create an array from a list or tuple
`np.zeros(shape)`	Create an array filled with 0s
`np.ones(shape)`	Create an array filled with 1s
`np.arange(start, stop, step)`	Create array with a range of numbers
`np.linspace(start, stop, num)`	Create array with `num` evenly spaced numbers between start and stop
`np.eye(n)`	Create identity matrix of size `n × n`
`np.random.rand(d1, d2, …)`	Create array with random floats in [0, 1)
`np.random.randint(low, high, size)`	Create array with random integers

---

2. Array Attributes

Attribute	Description
`.shape`	Returns dimensions of array

Attribute	Description
`.size`	Returns total number of elements
`.dtype`	Returns data type of elements
`.ndim`	Returns number of dimensions (axes)

---

3. Array Reshaping and Manipulation

Method	Description
`.reshape(shape)`	Change shape without changing data
`.flatten()`	Convert multi-dimensional array to 1D
`.ravel()`	Similar to flatten but returns view if possible
`.transpose()` or `.T`	Transpose array
`np.concatenate([a1, a2], axis)`	Join arrays along specified axis
`np.vstack([a1, a2])`	Stack arrays vertically
`np.hstack([a1, a2])`	Stack arrays horizontally
`np.split(a, indices_or_sections, axis)`	Split array into multiple sub-arrays

---

4. Indexing & Slicing

Method/Technique	Description
`arr[i]`	Access element at index `i`
`arr[start:end]`	Slice array from start to end-1
`arr[:, j]`	Access j-th column in 2D array
`arr[i, :]`	Access i-th row in 2D array
`arr[mask]`	Boolean indexing, e.g., `arr[arr>5]`

---

5. Mathematical Operations

Method	Description
`np.add(a, b)`	Element-wise addition
`np.subtract(a, b)`	Element-wise subtraction
`np.multiply(a, b)`	Element-wise multiplication
`np.divide(a, b)`	Element-wise division
`np.power(a, b)`	Element-wise exponentiation
`np.sqrt(a)`	Square root of each element
`np.exp(a)`	Exponential of each element
`np.log(a)`	Natural logarithm

Method	Description
`np.sum(a, axis)`	Sum of elements along axis
`np.mean(a, axis)`	Mean along axis
`np.std(a)`	Standard deviation
`np.min(a, axis)`	Minimum value
`np.max(a, axis)`	Maximum value
`np.argmin(a)`	Index of minimum value
`np.argmax(a)`	Index of maximum value

---

6. Linear Algebra

Method	Description
`np.dot(a, b)`	Dot product of arrays
`np.matmul(a, b)`	Matrix multiplication
`np.linalg.inv(a)`	Inverse of matrix
`np.linalg.det(a)`	Determinant
`np.linalg.eig(a)`	Eigenvalues and eigenvectors
`np.linalg.norm(a)`	Norm (magnitude) of vector/matrix

---

7. Random Number & Statistics

Method	Description
`np.random.rand(d1, d2, ...)`	Uniform random numbers [0,1)
`np.random.randn(d1, d2, ...)`	Normal distribution random numbers
`np.random.randint(low, high, size)`	Random integers
`np.random.choice(arr, size)`	Random sample from array
`np.unique(arr)`	Unique elements
`np.sort(arr)`	Sort array
`np.argsort(arr)`	Indices that would sort array
`np.where(condition)`	Indices where condition is True

---

8. Miscellaneous

Method	Description
`np.copy(a)`	Create a copy of array
`np.round(a, decimals)`	Round elements to given decimals
`np.clip(a, min, max)`	Limit values to a range
`np.isnan(a)`	Check for NaN values
`np.isfinite(a)`	Check for finite numbers

💡 **Tip:** Almost everything in NumPy works **vectorized**, meaning you can perform operations on the whole array **without loops**, which is faster and cleaner.

If you want, I can make a **quick one-page "cheat sheet" of the most frequently used NumPy methods with examples** so you can keep it as a reference while coding.

Do you want me to do that?

provide me code implementation for each method you gave

Absolutely! I'll give you a **Python script that demonstrates almost all the frequently used NumPy methods** I listed earlier. This will serve as a **complete hands-on guide**.

Python

```python
import numpy as np

print("===== 1. Array Creation =====")
# np.array
arr1 = np.array([1, 2, 3, 4, 5])
print("arr1:", arr1)

# np.zeros
zeros = np.zeros((2, 3))
print("zeros:\n", zeros)

# np.ones
ones = np.ones((3, 2))
print("ones:\n", ones)

# np.arange
arr_range = np.arange(0, 10, 2)
print("arr_range:", arr_range)

# np.linspace
linspace_arr = np.linspace(0, 1, 5)
print("linspace_arr:", linspace_arr)

# np.eye
identity = np.eye(3)
print("identity:\n", identity)

# Random numbers
```

```
rand_arr = np.random.rand(2, 3)
print("rand_arr:\n", rand_arr)

rand_int = np.random.randint(0, 10, 5)
print("rand_int:", rand_int)


print("\n===== 2. Array Attributes =====")
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print("arr2.shape:", arr2.shape)
print("arr2.size:", arr2.size)
print("arr2.dtype:", arr2.dtype)
print("arr2.ndim:", arr2.ndim)


print("\n===== 3. Reshaping & Manipulation =====")
arr = np.arange(1, 10)
reshaped = arr.reshape(3, 3)
print("reshaped:\n", reshaped)

flattened = reshaped.flatten()
print("flattened:", flattened)

raveled = reshaped.ravel()
print("raveled:", raveled)

transposed = reshaped.T
print("transposed:\n", transposed)

concat = np.concatenate([arr1, arr_range])
print("concatenate:", concat)

vstacked = np.vstack([arr1, arr_range])
print("vstacked:\n", vstacked)

hstacked = np.hstack([arr1.reshape(-1,1), arr_range.reshape(-1,1)])
print("hstacked:\n", hstacked)

split_arr = np.split(arr, 3)
print("split_arr:", split_arr)


print("\n===== 4. Indexing & Slicing =====")
arr = np.array([10, 20, 30, 40, 50])
print("arr[0]:", arr[0])
print("arr[-1]:", arr[-1])
print("arr[1:4]:", arr[1:4])
print("arr[:3]:", arr[:3])

arr2 = np.array([[1, 2, 3],
                 [4, 5, 6],
```

```
                [7, 8, 9]])
print("arr2[0,1]:", arr2[0,1])
print("arr2[:,1]:", arr2[:,1])
print("arr2[1,:]:", arr2[1,:])
print("Boolean indexing arr2>5:", arr2[arr2>5])


print("\n===== 5. Mathematical Operations =====")
a = np.array([1,2,3])
b = np.array([4,5,6])
print("add:", np.add(a,b))
print("subtract:", np.subtract(a,b))
print("multiply:", np.multiply(a,b))
print("divide:", np.divide(a,b))
print("power:", np.power(a,2))
print("sqrt:", np.sqrt(a))
print("exp:", np.exp(a))
print("log:", np.log(a))

arr = np.array([1,2,3,4,5])
print("sum:", np.sum(arr))
print("mean:", np.mean(arr))
print("std:", np.std(arr))
print("min:", np.min(arr))
print("max:", np.max(arr))
print("argmin:", np.argmin(arr))
print("argmax:", np.argmax(arr))


print("\n===== 6. Linear Algebra =====")
mat1 = np.array([[1,2],[3,4]])
mat2 = np.array([[5,6],[7,8]])
print("dot:", np.dot(mat1, mat2))
print("matmul:\n", np.matmul(mat1, mat2))
print("inverse:\n", np.linalg.inv(mat1))
print("determinant:", np.linalg.det(mat1))
eigvals, eigvecs = np.linalg.eig(mat1)
print("eigenvalues:", eigvals)
print("eigenvectors:\n", eigvecs)
print("norm:", np.linalg.norm(mat1))


print("\n===== 7. Random & Statistics =====")
rand_sample = np.random.choice([10,20,30,40], 3)
print("random choice:", rand_sample)

sorted_arr = np.sort(arr)
print("sorted:", sorted_arr)

argsorted_arr = np.argsort(arr)
print("argsort:", argsorted_arr)
```

```
unique_arr = np.unique([1,2,2,3,3,3])
print("unique:", unique_arr)

where_arr = np.where(arr>3)
print("where arr>3:", where_arr)


print("\n===== 8. Miscellaneous =====")
copy_arr = arr.copy()
print("copy:", copy_arr)

rounded = np.round(np.array([1.123, 2.567]), 1)
print("round:", rounded)

clipped = np.clip(np.array([1,5,10]), 3, 8)
print("clip:", clipped)

nan_check = np.isnan(np.array([1,np.nan,3]))
print("isnan:", nan_check)

finite_check = np.isfinite(np.array([1,np.inf,3]))
print("isfinite:", finite_check)
```

✅ **What this code covers:**

- Array creation, attributes, reshaping
- Indexing, slicing, and boolean indexing
- Math operations and statistical functions
- Linear algebra operations
- Random numbers and statistical methods
- Miscellaneous utilities like copy, clip, round, isnan

If you want, I can also make a **super short "1-page cheat sheet with 1-line example per method"**, so you can **quickly memorize and use it while coding**.

Do you want me to make that?

zeros = np.zeros((2, 3))
what is parameter

In the line:

```python
zeros = np.zeros((2, 3))
```

the **parameter** `(2, 3)` is the **shape of the array** you want to create. Let me break it down:

`np.zeros(shape, dtype=float, order='C')`

- `shape` → **required**
  - Specifies the **dimensions** of the array.
  - Can be an **integer** (for 1D) or a **tuple** (for 2D, 3D, …).
  - In your case: `(2, 3)` → 2 rows, 3 columns.
- `dtype` → optional
  - Data type of the elements. Default is `float`.
  - Example: `np.zeros((2,3), dtype=int)` → creates integers instead of floats.
- `order` → optional
  - `'C'` → row-major (C-style), `'F'` → column-major (Fortran-style). Usually left as default `'C'`.

Example:

```python
import numpy as np

# 2x3 array of zeros (default float)
zeros = np.zeros((2, 3))
print(zeros)
# Output:
# [[0. 0. 0.]
#  [0. 0. 0.]]

# 2x3 array of zeros (integers)
zeros_int = np.zeros((2, 3), dtype=int)
print(zeros_int)
# Output:
# [[0 0 0]
#  [0 0 0]]
```

So the **parameter** `(2, 3)` **defines the size of the array**.

If you want, I can make a **small table explaining the shape for 1D, 2D, 3D arrays in NumPy** — it's very handy to visualize.

Do you want me to do that?

> in python we can have while or for loop with else: so
> if the for loop didnt run even once due to condition and not break or continue so whether else block run of not

Ah! This is a subtle and often confusing behavior in Python. Let's go **step by step**.

Python `for` / `while` with `else`

- The `else` block **runs only if the loop completes normally**, **without hitting a** `break`.
- "Completes normally" means:
  - The loop **ran zero or more times** and **never encountered a** `break`.

Key Question:

> If the loop **doesn't run even once**, will the `else` run?

✅ **Yes!** The `else` block **still runs** if the loop didn't run at all, as long as there was no `break`.

Examples

1. `for` loop that doesn't run at all

```python
for i in range(0):  # range(0) → loop never runs
    print("Inside loop")
else:
    print("Else runs!")
```

Output:

```
Else runs!
```

> Explanation: Loop didn't run, no `break` happened → `else` executes.

---

2. `for` loop with `break`

```python
for i in range(5):
    if i == 2:
        break
else:
    print("Else runs!")
```

Output:

```
# nothing
```

> Explanation: Loop ran, but `break` happened → `else` does **not run**.

---

3. `while` loop that doesn't run at all

```python
x = 10
while x < 5:  # condition false from the start
    print("Inside while")
else:
    print("Else runs!")
```

Output:

```
Else runs!
```

> Explanation: `while` didn't run even once, no `break` → `else` executes.

Summary Table

Loop Type	Runs?	Break?	Else Executes?
`for` / `while` ran 0 times	No	No	✅ Yes
`for` / `while` ran n>0 times	Yes	No	✅ Yes
`for` / `while` ran n>0 times	Yes	Yes	❌ No

So the **rule of thumb:**

> `else` **runs if the loop wasn't broken, even if it never ran at all.**

If you want, I can make a **visual diagram showing when** `for/while else` **executes** — it makes this very easy to remember.

Do you want me to make it?

collection framework and method of each in python

Sure! In Python, the **collection framework** is not exactly like Java, but Python provides several **built-in collection types** (data structures) that are highly used, each with its own **methods and functionalities**. I'll explain **types, examples, and frequently used methods**.

---

1. List

- **Description:** Ordered, mutable, allows duplicates.
- **Syntax:** `my_list = [1, 2, 3]`

Common Methods

Method	Description	Example
`append(x)`	Add `x` at the end	`lst.append(4)`
`extend(iterable)`	Add multiple elements	`lst.extend([5,6])`
`insert(i, x)`	Insert `x` at index `i`	`lst.insert(1, 9)`
`remove(x)`	Remove first occurrence of `x`	`lst.remove(2)`
`pop([i])`	Remove and return element at `i` (last if not specified)	`lst.pop()`
`clear()`	Remove all elements	`lst.clear()`
`index(x)`	Index of first occurrence of `x`	`lst.index(3)`
`count(x)`	Count occurrences of `x`	`lst.count(2)`
`sort()`	Sort in-place	`lst.sort()`
`reverse()`	Reverse in-place	`lst.reverse()`
`copy()`	Return shallow copy	`lst2 = lst.copy()`

Example:

Python

```python
lst = [1, 3, 2]
lst.append(4)
lst.sort()
print(lst)  # [1, 2, 3, 4]
```

---

2. Tuple

- **Description:** Ordered, immutable, allows duplicates.
- **Syntax:** `my_tuple = (1, 2, 3)`

Common Methods

Method	Description	Example
`count(x)`	Count occurrences of `x`	`(1,2,2).count(2)` → 2
`index(x)`	Index of first occurrence of `x`	`(1,2,3).index(2)` → 1

Note: Since tuples are immutable, no methods like `append` or `remove`.

---

3. Set

- **Description:** Unordered, mutable, no duplicates.
- **Syntax:** `my_set = {1, 2, 3}`

Common Methods

Method	Description	Example
`add(x)`	Add element `x`	`s.add(4)`
`remove(x)`	Remove element `x` (error if not exists)	`s.remove(3)`
`discard(x)`	Remove element `x` (no error if not exists)	`s.discard(5)`
`pop()`	Remove and return an arbitrary element	`s.pop()`
`clear()`	Remove all elements	`s.clear()`
`union(s2)`	Union with another set	`s.union({4,5})`
`intersection(s2)`	Common elements	`s.intersection({2,3})`
`difference(s2)`	Elements in s but not in s2	`s.difference({2})`
`issubset(s2)`	Check subset	`s.issubset({1,2,3,4})`

Method	Description	Example
`issuperset(s2)`	Check superset	`s.issuperset({1})`

Example:

```python
s = {1, 2, 3}
s.add(4)
print(s)  # {1,2,3,4}
print(s.intersection({2,5}))  # {2}
```

---

4. Dictionary

- **Description:** Key-value pairs, unordered (Python 3.7+ preserves insertion order), mutable.
- **Syntax:** `my_dict = {'a': 1, 'b': 2}`

Common Methods

Method	Description	Example
`keys()`	Returns keys	`d.keys()`
`values()`	Returns values	`d.values()`
`items()`	Returns (key, value) tuples	`d.items()`
`get(key, default)`	Get value for key (default if not found)	`d.get('c', 0)`
`pop(key)`	Remove key and return value	`d.pop('a')`
`popitem()`	Remove and return last inserted item	`d.popitem()`
`update(other_dict)`	Add/update key-value pairs	`d.update({'d': 4})`
`clear()`	Remove all items	`d.clear()`
`setdefault(key, default)`	Return value if exists, else set to default	`d.setdefault('e', 5)`

Method	Description	Example
`copy()`	Shallow copy	`d2 = d.copy()`

Example:

```python
d = {'a': 1, 'b': 2}
d['c'] = 3
d.update({'b': 5})
print(d)  # {'a':1, 'b':5, 'c':3}
```

---

5. Other Collection Utilities (from `collections` module)

Python also has **specialized collection types** in the `collections` module:

Collection	Description
`deque`	Double-ended queue, fast append/pop from both ends
`Counter`	Counts occurrences of elements
`defaultdict`	Dictionary with default values for missing keys
`OrderedDict`	Dict that remembers insertion order (useful < Python3.7)
`namedtuple`	Tuple with named fields

Example (Counter):

```python
from collections import Counter
lst = [1,2,2,3,3,3]
c = Counter(lst)
print(c)  # Counter({3:3, 2:2, 1:1})
```

✅ Summary

- **List** → ordered, mutable, allows duplicates
- **Tuple** → ordered, immutable, allows duplicates
- **Set** → unordered, mutable, no duplicates
- **Dictionary** → key-value pairs, mutable
- **Special collections** → `deque`, `Counter`, `defaultdict`, `namedtuple`

If you want, I can also make a **Python "Collections Cheat Sheet" with all methods + small example for each**, so you can **use it as a quick reference while coding**.

Do you want me to make that?

FIND FIRST 3 LARGEST ELEEMNT IN LIST ON PYTHON

Sure! There are **multiple ways** to find the first 3 largest elements in a list in Python. I'll explain **3 common approaches** with code.

---

1. Using `sorted()` (simple and readable)

```python
lst = [5, 1, 9, 3, 7, 2]

# Sort in descending order and take first 3
largest_three = sorted(lst, reverse=True)[:3]
print(largest_three)  # Output: [9, 7, 5]
```

Explanation:

- `sorted(lst, reverse=True)` → sorts the list from largest to smallest
- `[:3]` → slices the first 3 elements

✅ Works for small/medium lists.

---

2. Using `heapq.nlargest()` (efficient for large lists)

```python
import heapq
```

```
lst = [5, 1, 9, 3, 7, 2]

largest_three = heapq.nlargest(3, lst)
print(largest_three)  # Output: [9, 7, 5]
```

Explanation:

- `heapq.nlargest(n, iterable)` → finds the `n` largest elements efficiently using a heap
- Faster than sorting if the list is very large

---

3. Using a loop (manual method)

```python
lst = [5, 1, 9, 3, 7, 2]
largest = []

for _ in range(3):
    max_val = max(lst)
    largest.append(max_val)
    lst.remove(max_val)  # Remove it so next max can be found

print(largest)  # Output: [9, 7, 5]
```

Explanation:

- Use `max()` to find the largest element
- Remove it and repeat 3 times

⚠️ Modifies the original list.

💡 **Tip:**

- Use `sorted()` or `heapq.nlargest()` in real applications; they are cleaner and faster.
- If you **want to keep the original list unchanged**, `heapq.nlargest()` is the safest choice.

If you want, I can also show a **one-liner using Python `set()` to get 3 largest unique elements** — useful when the list has duplicates.

Do you want me to show that?

In Python, a **dictionary** stores data as **key-value pairs**, like this:

```python
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

To **access keys**, there are several ways:

---

1. Using `.keys()` method

```python
keys = my_dict.keys()
print(keys)          # dict_keys(['a', 'b', 'c'])
print(list(keys))    # ['a', 'b', 'c'] → convert to list if needed
```

- `.keys()` returns a **view object** (`dict_keys`) that behaves like a set.
- You can **iterate** over it in loops:

```python
for key in my_dict.keys():
    print(key)
```

Output:

```
a
b
c
```

---

2. Iterating over the dictionary directly

```python
for key in my_dict:
    print(key)
```

- By default, iterating a dictionary gives its **keys**.
- Output is same as above.

3. Using list comprehension (optional)

```python
keys_list = [key for key in my_dict]
print(keys_list)  # ['a', 'b', 'c']
```

✅ Summary

Method	Notes
`my_dict.keys()`	Returns view object of keys
`for key in my_dict:`	Iterates over keys directly
`[key for key in my_dict]`	Creates a list of keys using comprehension

If you want, I can also show how to **access keys and values together**, which is very commonly used in Python.

Do you want me to show that?

AND FOR VALUES

Absolutely! Accessing **values** in a Python dictionary is just as simple as accessing keys. Here's a clear explanation.

Suppose we have:

```python
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

---

1. Using `.values()` method

```python
values = my_dict.values()
print(values)          # dict_values([1, 2, 3])
print(list(values))    # [1, 2, 3] → convert to list if needed
```

- `.values()` returns a **view object** (`dict_values`) containing all the values.
- You can **iterate** over it:

```python
for value in my_dict.values():
    print(value)
```

Output:

```
1
2
3
```

---

2. Using key access

```python
for key in my_dict:
    print(my_dict[key])
```

- Here, you loop over keys, and then access each value using `my_dict[key]`.

---

3. Using list comprehension

```python
values_list = [my_dict[key] for key in my_dict]
print(values_list)  # [1, 2, 3]
```

✅ Summary

Method	Notes
`my_dict.values()`	Returns a view object of all values
`for key in my_dict: my_dict[key]`	Access values via keys
`[my_dict[key] for key in my_dict]`	Creates a list of values using comprehension

If you want, I can also explain **how to access both keys and values together** efficiently, which is usually done with `.items()`.

Do you want me to show that?